

Module 2.5: SOP and POS forms of Boolean Expressions

Session 2.5: Focus

- Simplification Techniques
- Simplification using Boolean Algebra
 - Quiz 1 to 3
- Two forms of Boolean Expressions
 - Sum-of-Products (minterms)
 - Products-of-Sums (maxterms)
 - Canonical notation
 - Example
- Home Work

Introduction to Boolean Algebra

Recap: Variables, Literals and Terms

- **Variables** are different symbols in a **Boolean expression**
- They may take on the values '0' or '1'
- **Complement** of a variable is **not** considered as a **separate variable**
- Each occurrence of a **variable** or its **complement** is called a **literal**
- A **term** is the **expression formed by literals and operations at one level.**
- Each **term** requires a **gate** and each **variable within the term** designates an **input to the gate**

$$\overline{A} + A.B$$

Variables : 2

Literals : 3

Terms : 2

Simplification Techniques

Need for Simplification

- Simplification means simplifying or more precisely minimizing a given complex Boolean expression
- The **primary objective** of all simplification procedures is to **obtain** an expression that has the **minimum number of terms**
 - A simplified Boolean expression uses the fewest gates possible to implement a given expression
 - If gates are reduced, the complexity of the implementation as well as power consumption are also reduced
- Obtaining an expression with the **minimum** number of **literals** or **symbols** is usually the **secondary objective**

Simplification Techniques

- Simplification using Boolean algebra
- Other two popular methods are:
 - The Quine–McCluskey tabular method;
 - The Karnaugh map method

Simplification Using Boolean Algebra

Simplification Using Boolean Algebra

- The approach taken in this section is to
 - Use the basic laws, rules and theorems of Boolean Algebra
 - To manipulate and simplify an Expression
 - Which will result in less number of gates/literals without changing the function of the original Boolean Expression
- A **simplified** Boolean Expression **uses the fewest gates** possible **to implement** a given expression

Quiz 1 : Simplification Using Boolean Algebra

- Simplify the expression: $AB + A(B+C) + B(B+C)$
- **Step 1:** Apply Distributive law to 2nd and 3rd items
 - $AB + \mathbf{AB} + \mathbf{AC} + \mathbf{BB} + \mathbf{BC}$
- **Step 2:** Apply **rule 7** ($BB = B$)
 - $AB + AB + AC + \mathbf{B} + BC$
- **Step 3:** Apply **rule 5** ($AB + AB = AB$)
 - $\mathbf{AB} + AC + B + BC$
- **Step 4:** Apply **rule 10** ($B + BC = B$)
 - $AB + AC + \mathbf{B}$
- **Step 5:** Apply **rule 10** ($AB + B = B$)
 - $\mathbf{AC + B}$

Note: The **terms in Bold** are the ones **affected** by the **rules** in **each step**.

Quiz 2 : Simplify the Boolean Expression

- Simplify the expression: $[A\bar{B}(C + BD) + \bar{A}\bar{B}]C$
- **Step 1:** Apply Distributive law to the terms within brackets
 - $(A\bar{B}C + A\bar{B}BD + \bar{A}\bar{B})C$
- **Step 2:** Apply **rule 8** ($B\bar{B} = 0$)
 - $(A\bar{B}C + A \cdot 0 \cdot D + \bar{A}\bar{B})C$
- **Step 3:** Apply **rule 3** ($A \cdot 0 \cdot D = 0$)
 - $(A\bar{B}C + 0 + \bar{A}\bar{B})C$
- **Step 4:** Apply **rule 1** (drop the 0)
 - $(A\bar{B}C + \bar{A}\bar{B})C$
- **Step 5:** Apply Distributive law
 - $A\bar{B}CC + \bar{A}\bar{B}C$

Quiz 2 ... contd

- Simplify the expression: $[\overline{A}\overline{B}(C + BD) + \overline{A}\overline{B}]C$
- From Step 5: $\overline{A}\overline{B}CC + \overline{A}\overline{B}\overline{C}C$
- Step 6: Apply rule 7 ($CC = C$) to the first term
 - $\overline{A}\overline{B}C + \overline{A}\overline{B}\overline{C}$
- Step 7: Factor out $\overline{B}C$
 - $\overline{B}C(A + \overline{A})$
- Step 8: Apply rule 6 ($A + \overline{A} = 1$)
 - $\overline{B}C \cdot 1$
- Step 9: Apply rule 4 (drop the 1)
 - $\overline{B}C$

Quiz 3 : Simplify the Boolean Expression

- Simplify the expression: $\overline{AB + AC + A\overline{B}C}$
- **Step 1:** Apply DeMorgon's theorem to the first term
 - $(\overline{AB})(\overline{AC}) + \overline{A\overline{B}C}$
- **Step 2:** Apply DeMorgon's theorem to each term in parenthesis
 - $(\overline{A} + \overline{B})(\overline{A} + \overline{C}) + \overline{A\overline{B}C}$
- **Step 3:** Apply Distributive law to the terms within parenthesis
 - $\overline{A}\overline{A} + \overline{A}\overline{C} + \overline{A}\overline{B} + \overline{B}\overline{C} + \overline{A\overline{B}C}$
- **Step 4:** Apply **rule 7** ($\overline{A}\overline{A} = \overline{A}$) to the first term
 - $\overline{A} + \overline{A}\overline{C} + \overline{A}\overline{B} + \overline{B}\overline{C} + \overline{A\overline{B}C}$
- **Step 5:** Apply **rule 10** ($A + A\overline{B} = A$) to the 3rd and last terms
 - $\overline{A} + \overline{A}\overline{C} + \overline{A}\overline{B} + \overline{B}\overline{C}$

Quiz 3 ... contd

- From **Step 5**: $\bar{A} + \bar{A}\bar{C} + \bar{A}\bar{B} + \bar{B}\bar{C}$
- **Step 6**: Apply **rule 10** ($A + AB = A$) to the 1st and 2nd terms
 - $\bar{A} + \bar{A}\bar{B} + \bar{B}\bar{C}$
- **Step 7**: Apply **rule 10** ($A + AB = A$) to the 1st and 2nd terms
 - $\bar{A} + \bar{B}\bar{C}$

Two Forms of Boolean Expressions

Two Forms of Boolean Expressions

- Boolean Expressions are normally expressed in one of these forms for Simplification
 - **Sum-of-Products (SOP)**
 - **Product-of-Sums (POS)**
- Let us see them before studying other Simplification techniques

Sum of Products

- **Sum-of-products** expression contains the **sum** of different **terms**, with each term being either a **single literal** or a **product of more than one literal**
- It can be obtained from the **truth table** directly by considering those **input combinations** that produce a **logic '1'** at the **output**.
- Each such input combination produces a term, which are called **minterms**
- A **minterm** is a combination of variables that produces a **1** in the function and then taking the **OR** of **all those terms**

Example: Sum-of-Products

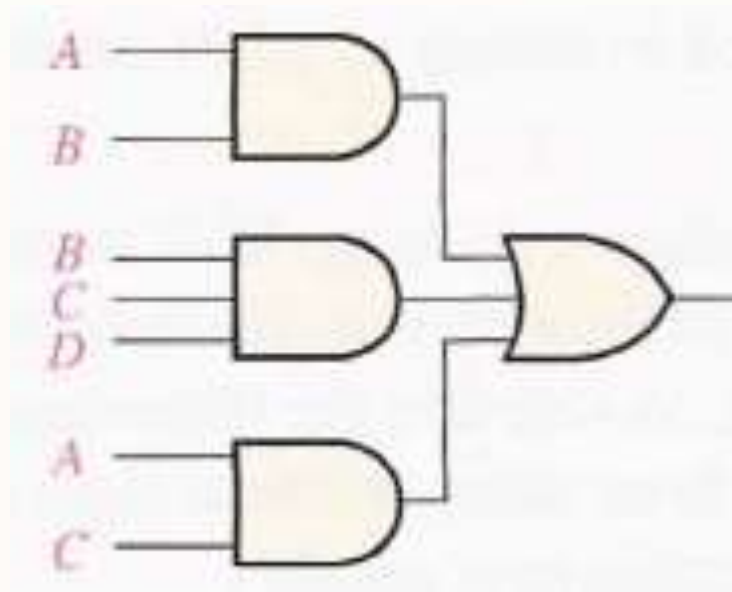
- For Example **Truth table** below can be expressed as:
 - Only those terms which produce an **output** of **1**

<i>A</i>	<i>B</i>	<i>C</i>	<i>Y</i>
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

$$Y = \overline{A} . \overline{B} . \overline{C} + \overline{A} . B . C + A . B . \overline{C} + A . \overline{B} . C$$

Implementation of Sum of Products

- **Example:** $AB + BCD + AC$ can be implemented as shown below:



AND-OR

Quiz 4: Is this in SOP or POS form?

- $\overline{ABC} + A\overline{BC}$: **Neither in SOP nor POS form!!**

For an expression to properly follow the **SOP** or **POS** canonical form, **no complementation bar** should **cover more than one variable!**

Minterms of Three Binary Variables

- All the combinations of Sum-of-Products of three variables and their nomenclature are given below

<i>x</i>	<i>y</i>	<i>z</i>	Minterms	
			Term	Designation
0	0	0	$x'y'z'$	m_0
0	0	1	$x'y'z$	m_1
0	1	0	$x'yz'$	m_2
0	1	1	$x'yz$	m_3
1	0	0	$xy'z'$	m_4
1	0	1	$xy'z$	m_5
1	1	0	xyz'	m_6
1	1	1	xyz	m_7

Product-of-Sums

- **Product-of-Sums** expression contains the **product** of different **terms**, with each term being either a **single literal** or a **sum of more than one literal**
- It can be obtained from the **truth table** directly by considering those **input combinations** that produce a **logic '0'** at the **output**.
- Each such input combination produces a term, which are called **maxterms**
- A **maxterm** is a combination of variables that produces a **0** in the function and then taking the **AND** of **all those terms**
- Boolean functions expressed as a **sum of minterms** or **product of maxterms** are said to be in **canonical form**

Example: Product-of-Sums

- For Example **Truth table** below can be expressed as:

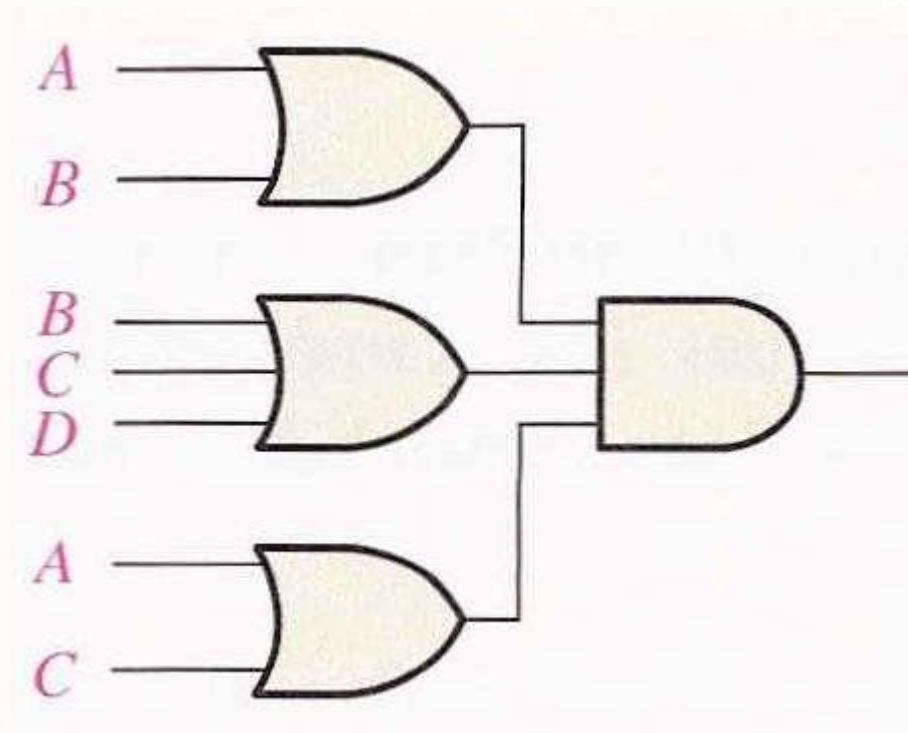
<i>A</i>	<i>B</i>	<i>C</i>	<i>Y</i>
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

$$Y = (A + B + \overline{C}).(A + \overline{B} + C).(\overline{A} + B + C).(\overline{A} + \overline{B} + \overline{C})$$

Implementation of Product-of-Sums

- **Example:** $(A + B) \cdot (B + C + D) \cdot (A + C)$

OR-AND



Maxterms of Three Binary Variables

- All the combinations of Product-of-Sums of three variables and their nomenclature are given below



Minterms							Maxterms	
Term	Designation	x	y	z	Term	Designation		
$x'y'z'$	m_0	0	0	0	$x + y + z$	M_0		
$x'y'z$	m_1	0	0	1	$x + y + z'$	M_1		
$x'yz'$	m_2	0	1	0	$x + y' + z$	M_2		
$x'yz$	m_3	0	1	1	$x + y' + z'$	M_3		
$xy'z'$	m_4	1	0	0	$x' + y + z$	M_4		
$xy'z$	m_5	1	0	1	$x' + y + z'$	M_5		
xyz'	m_6	1	1	0	$x' + y' + z$	M_6		
xyz	m_7	1	1	1	$x' + y' + z'$	M_7		

Another Example

$$f_1 = x'y'z + xy'z' + xyz = m_1 + m_4 + m_7$$

$$f_2 = x'yz + xy'z + xyz' + xyz = m_3 + m_5 + m_6 + m_7$$

x	y	z	Function f_1	Function f_2
0	0	0	0	0
0	0	1	1	0
0	1	0	0	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$$\begin{aligned} f_1 &= (x + y + z)(x + y' + z)(x' + y + z')(x' + y' + z) \\ &= M_0 \cdot M_2 \cdot M_3 \cdot M_5 \cdot M_6 \end{aligned}$$

$$\begin{aligned} f_2 &= (x + y + z)(x + y + z')(x + y' + z)(x' + y + z) \\ &= M_0 M_1 M_2 M_4 \end{aligned}$$

Home Work

- **Simplify** the following Boolean Expressions, using Boolean Algebra:

- 1. $\bar{A}BC + A\bar{B}\bar{C} + \bar{A}\bar{B}\bar{C} + A\bar{B}C + ABC$

- Solution: $AC + BC + \bar{B}\bar{C}$ or $BC + A\bar{B} + \bar{B}\bar{C}$

- 2. $xy + x'z + yz$

- Solution: $xy + x'z$.

- 3. $(x + y)(x + y')$

- Solution: x

Note: For the problem 1, both the above solutions are correct.

You can verify this by constructing a truth table.

The reason for getting different simplified equations is the order we apply the rules and the terms that we take up first.

Session 2.5: Summary

- Simplification Techniques
- Simplification using Boolean Algebra
 - Quiz 1 to 3
- Two forms of Boolean Expressions
 - Sum-of-Products (minterms)
 - Products-of-Sums (maxterms)
 - Canonical notation
 - Example
- Home Work

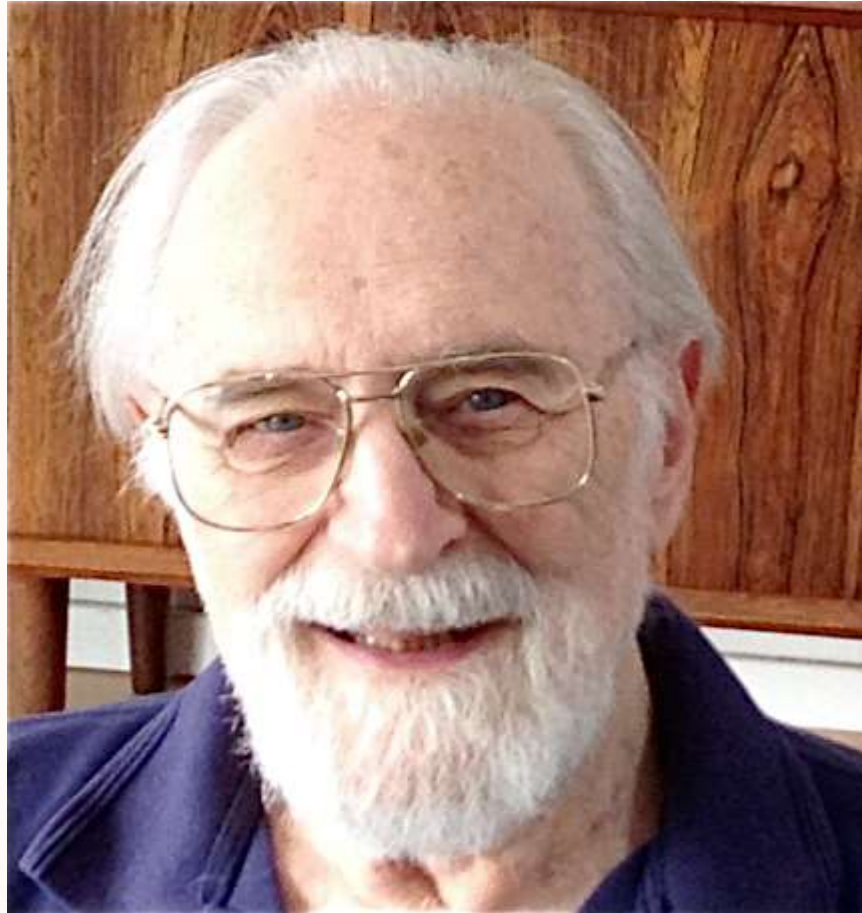
Module 2.5: Introduction to Karnaugh map

Session 2.6: Focus

- Introduction to Karnaugh map
- Examples
 - SOP form
 - POS form
 - Both SOP and POS
 - Implementation
- Don't cares in K-map
 - Example

Karnaugh map Simplification Method

Maurice Karnaugh

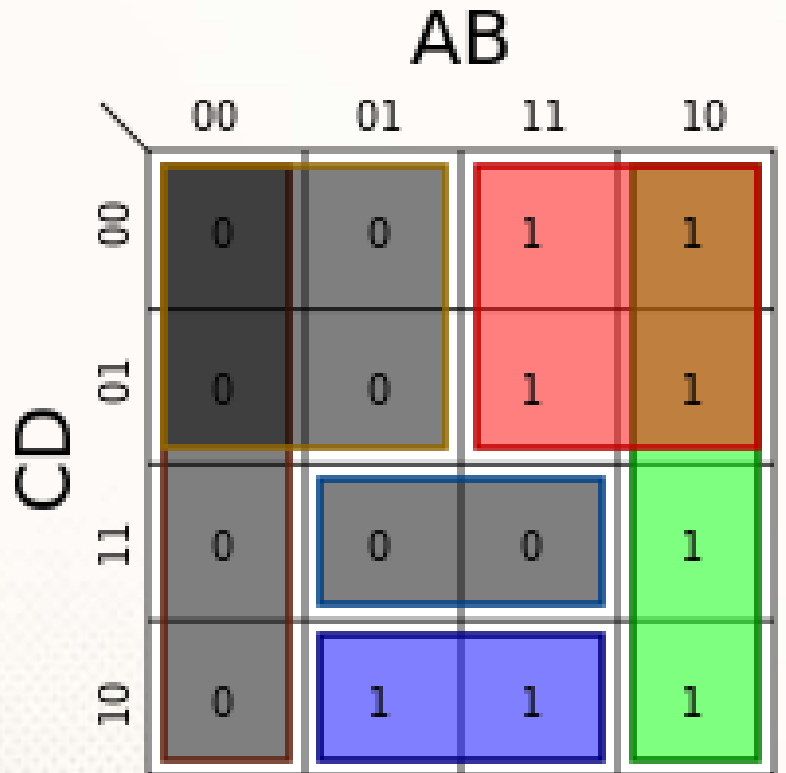


Maurice Karnaugh (October 4, 1924 – Nov 8, 2022) is an American physicist and mathematician known for the Karnaugh map used in Boolean algebra.

Karnaugh Map

- The **Karnaugh map**, also known as the **K-map**, is a method to simplify Boolean algebra expressions.
- **Maurice Karnaugh** introduced it in **1953**
- It **reduces** the need for **extensive calculations** by taking advantage of humans' **pattern recognition** capability
- The required Boolean results are **transferred from a truth table** onto **a two-dimensional grid**
 - Where the cells are **ordered** in **Gray code**, and
 - **Each cell** position represents **one combination of input conditions**
 - while each **cell value** represents the corresponding **output** value

Sample: Karnaugh Map - corrected



$$f(A,B,C,D) = \Sigma(6,8,9,10,11,12,13,14)$$

$$F = AC' + AB' + BCD'$$

$$F = (A+B)(A+C)(B'+C'+D')$$

This image actually shows
Two Karnaugh maps:
for the **function f**,
Using **minterms**
(**colored rectangles**)
and
for its **complement f'**,
using **maxterms**
(**gray rectangles**).

[Courtesy: Wikipedia](#)

Why Gray Code in K-map?

- The variable combinations in the K-map in the adjacent cells **differ only by one bit**, by ordering them using **Gray codes**
- This helps in simplifying them by combining the adjacent elements if both output **one** (for minterms) or **zero** (for maxterms) since they differ by only one value

Example 1 (SOP)

$$\text{Out} = \overline{A}\overline{B}CD + \overline{A}BCD + ABCD + A\overline{B}CD + AB\overline{C}\overline{D} + AB\overline{C}D + ABC\overline{D}$$

		CD			
		00	01	11	10
A B	00			1	
	01			1	
	11	1	1	1	1
	10			1	

Out = AB + CD

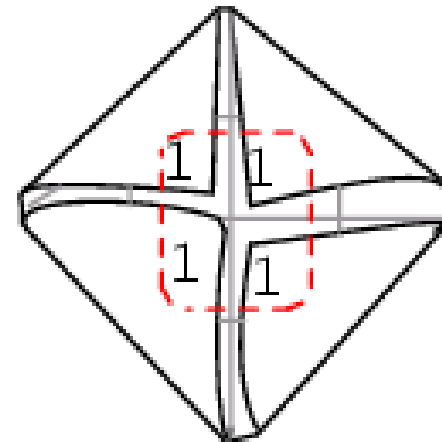
Groups must be a power of 2.

Example 2 (SOP)

$$\text{Out} = \overline{A}\overline{B}\overline{C}\overline{D} + \overline{A}\overline{B}C\overline{D} + A\overline{B}\overline{C}\overline{D} + A\overline{B}C\overline{D}$$

		CD			
		00	01	11	10
A \ B	00	1			1
	01				
	11				
	10	1			1

$$\text{Out} = \overline{B}\overline{D}$$



Example 3 (SOP)

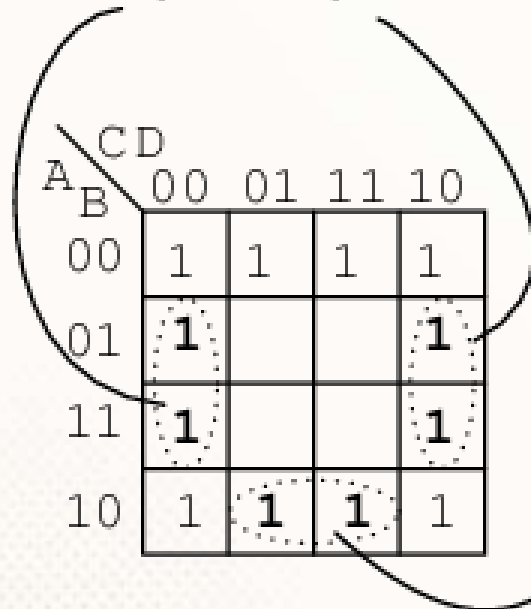
$$\text{Out} = \overline{A}\overline{B}\overline{C}\overline{D} + \overline{A}\overline{B}\overline{C}D + \overline{A}\overline{B}C\overline{D} + \overline{A}\overline{B}CD \\ + A\overline{B}\overline{C}\overline{D} + A\overline{B}\overline{C}D + A\overline{B}C\overline{D} + A\overline{B}CD$$

		CD			
		00	01	11	10
A \ B	00	1	1	1	1
	01				
	11				
	10	1	1	1	1

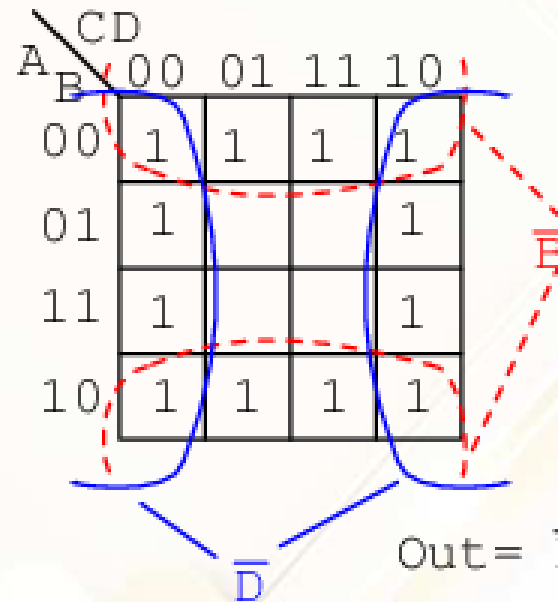
$$\text{Out} = \overline{B}$$

Example 4 (SOP)

$$\text{Out} = \overline{A}\overline{B}\overline{C}\overline{D} + \overline{A}\overline{B}\overline{C}D + \overline{A}\overline{B}C\overline{D} + \overline{A}\overline{B}CD + B\overline{C}\overline{D} + B\overline{C}D + A\overline{B}\overline{C}\overline{D} + A\overline{B}D + A\overline{B}C\overline{D}$$



		CD			
		00	01	11	10
A \ B	00	1	1	1	1
	01	1			1
	11	1			1
	10	1	1	1	1



		CD			
		00	01	11	10
A \ B	00	1	1	1	1
	01	1			1
	11	1			1
	10	1	1	1	1

$\overline{D}$ $\overline{B}$

$$\text{Out} = \overline{B} + \overline{D}$$

Example 5 (SOP)

$$\text{Out} = \overline{A}\overline{B}\overline{C}\overline{D} + \overline{A}\overline{B}C\overline{D} + A\overline{B}\overline{C}\overline{D} + ABCD$$

		CD					
		00	01	11	10		
A B	00	1			1		
	01						
	11			1			
	10	1					

$$\text{Out} = \overline{B}\overline{C}\overline{D} + \overline{A}\overline{B}\overline{D} + ABCD$$

Example 6 (SOP)

$$\text{Out} = \overline{A}\overline{B}\overline{C}\overline{D} + \overline{A}\overline{B}\overline{C}D + \overline{A}\overline{B}C\overline{D} + \overline{A}BCD + ABCD + ABC\overline{D} + A\overline{B}C\overline{D} + A\overline{B}C\overline{D}$$

A \ B \ CD	00	01	11	10
00	1	1		
01		1	1	
11			1	1
10	1			1

A \ B \ CD	00	01	11	10
00	1	1		
01		1	1	
11			1	1
10	1			1

$$\text{Out} = \overline{B}\overline{C}\overline{D} + \overline{A}\overline{C}D + BCD + ACD$$

$$\text{Out} = \overline{A}\overline{B}\overline{C} + \overline{A}BD + ABC + A\overline{B}\overline{D}$$

Example 7 (SOP)

$$\begin{aligned}
 \text{Out} = & \overline{A}\overline{B}\overline{C}\overline{D} + \overline{A}\overline{B}\overline{C}D + \overline{A}\overline{B}C\overline{D} \\
 & + \overline{A}B\overline{C}\overline{D} + \overline{A}B\overline{C}D + \overline{A}BC\overline{D} \\
 & + AB\overline{C}\overline{D} + AB\overline{C}D + ABCD
 \end{aligned}$$

A \ B \ C D		C D			
		00	01	11	10
A \ B	00	1	1	1	
	01	1	1	1	
	11	1	1	1	
	10				

A \ B \ C D		C D			
		00	01	11	10
A \ B	00	1	1	1	
	01	1	1	1	
	11	1	1	1	
	10				

A \ B \ C D		C D			
		00	01	11	10
A \ B	00	1	1	1	
	01	1	1	1	
	11	1	1	1	
	10				

$$\text{Out} = \overline{A}\overline{C} + \overline{A}D + B\overline{C} + BD$$

Example 8 (SOP)

$$\text{Out} = \overline{A}\overline{B}\overline{C}\overline{D} + \overline{A}\overline{B}CD + \overline{A}B\overline{C}\overline{D} + \overline{A}B\overline{C}D + A\overline{B}\overline{C}\overline{D} + A\overline{B}\overline{C}D + ABC\overline{D} + ABCD + A\overline{B}CD + A\overline{B}CD$$

A \ B		CD			
		00	01	11	10
00	1			1	
01	1			1	
11	1	1		1	
10	1			1	

$$\text{Out} = \overline{C}\overline{D} + CD + A\overline{B}\overline{C}$$

A \ B		CD			
		00	01	11	10
00	1			1	
01	1			1	
11	1	1		1	
10	1			1	

$$\text{Out} = \overline{C}\overline{D} + CD + ABD$$

Example 9 (SOP)

$$\text{Out} = \bar{C} + ABCD$$

		CD			
		00	01	11	10
AB	00	1	1		
	01	1	1		
	11	1	1	1	
	10	1	1		

$$\text{Out} = \bar{C} + ABD$$

Simplification by Boolean Algebra

$$\text{Out} = \bar{C} + ABCD$$

Applying rule $A + \bar{A}B = A + B$ to the $\bar{C} + ABCD$ term

$$\text{Out} = \bar{C} + ABD$$

SOP and POS Forms

SOP

$$F = B'D' + B'C' + A'C'D$$

Combining 0 terms yields:

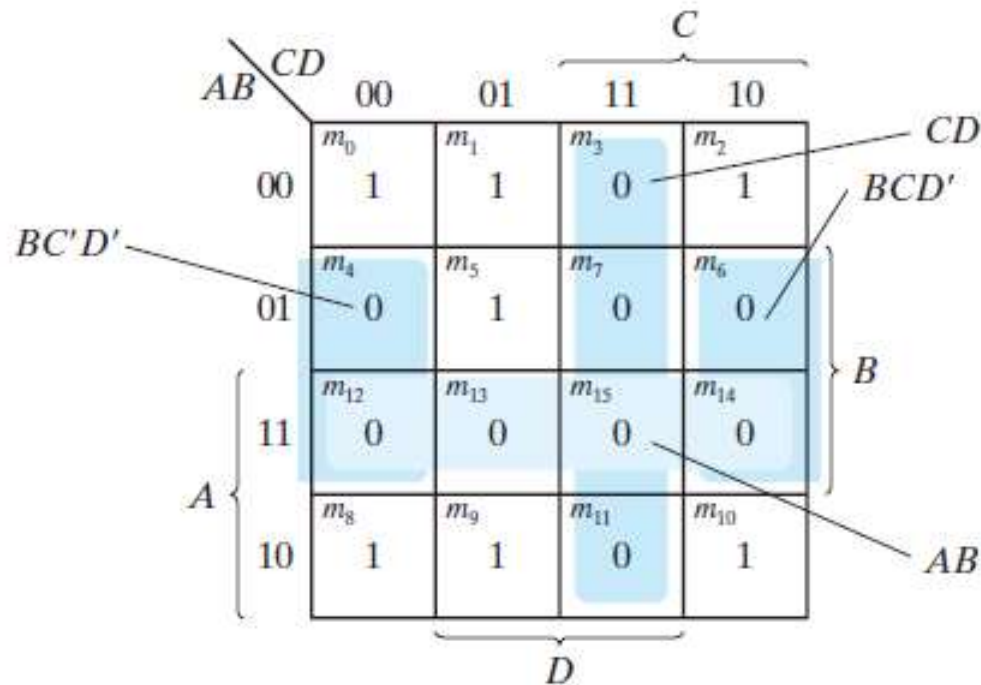
$$F' = AB + CD + BD'$$

Applying DeMorgan's theorem

Complement of F' is F

POS

$$F = (A' + B')(C' + D')(B' + D)$$



		C			
	CD	00	01	11	10
AB	00	m_0 1	m_1 1	m_3 0	m_2 1
	01	m_4 0	m_5 1	m_7 0	m_6 0
	11	m_{12} 0	m_{13} 0	m_{15} 0	m_{14} 0
	10	m_8 1	m_9 1	m_{11} 0	m_{10} 1

Example 1 (POS)

Out = (A + B + C)
Maxterm = A + B + C
Numeric = 1 1 1
Complement = 0 0 0

A \ BC	00 01 11 10			
	00	01	11	10
0	0	1	1	1
1	1	1	1	1

Example 2 (POS)

$$\begin{aligned}\text{Out} &= (\overline{A} + \overline{B} + \overline{C}) \\ \text{Maxterm} &= \overline{A} + \overline{B} + \overline{C} \\ \text{Numeric} &= 0 \quad 0 \quad 0 \\ \text{Complement} &= 1 \quad 1 \quad 1\end{aligned}$$

A \ BC	BC			
	00	01	11	10
0	1	1	1	1
1	1	1	0	1

Example 3 (POS) ... contd in the next slide

$$\text{Out} = (A + B + C) (A + B + \bar{C})$$

$$\text{Maxterm} = (A + B + C) \quad \text{Maxterm} = (A + B + \bar{C})$$

$$\text{Numeric} = 1 \ 1 \ 1 \quad \text{Numeric} = 1 \ 1 \ 0$$

$$\text{Complement} = 0 \ 0 \ 0 \quad \text{Complement} = 0 \ 0 \ 1$$

		BC			
A		00	01	11	10
0		0	0	1	1
1		1	1	1	1

Example 3 (POS) ... contd

$$\text{Out} = (A + B + C) (A + B + \overline{C})$$

A \ BC				
	00	01	11	10
0	0	0	1	1
1	1	1	1	1

A B C = 0 0 X

Complement = 1 1 X

Sum-term = (A + B)

Out = (A + B)

Quiz 1:

Simplify the Boolean expression using K-map

$$\text{Out} = (A+B+C+\bar{D})(A+B+\bar{C}+D)(A+\bar{B}+C+\bar{D})(A+\bar{B}+\bar{C}+D) \\ (\bar{A}+\bar{B}+\bar{C}+D)(\bar{A}+B+C+\bar{D})(\bar{A}+B+\bar{C}+D)$$

$$\text{Out} = (A+B+C+\bar{D})(A+B+\bar{C}+D)(A+\bar{B}+C+\bar{D})(A+\bar{B}+\bar{C}+D) \\ (\bar{A}+\bar{B}+\bar{C}+D)(\bar{A}+B+C+\bar{D})(\bar{A}+B+\bar{C}+D)$$

		CD			
		00	01	11	10
AB	00		0		0
	01		0		0
	11				0
	10		0		0

Quiz 1 : Solution

		CD				input		complement	Sum-term
A	B	00	01	11	10				
						ABCD =			
00			0		0	X001	> X110	>	$(B + C + \bar{D})$
01			0		0	0X01	> 1X10	>	$(A + C + \bar{D})$
11					0	XX10	> XX01	>	$(\bar{C} + D)$
10			0		0				

Out = $(B + C + \bar{D})$ $(A + C + \bar{D})$ $(\bar{C} + D)$

Quiz 2:

Simplify the **Product-Of-Sums** Boolean expression below, providing a **result** in **SOP** form

$$\text{Out} = (A + B + C + \bar{D}) (A + B + \bar{C} + D) (A + \bar{B} + C + \bar{D}) (A + \bar{B} + \bar{C} + D) (\bar{A} + \bar{B} + \bar{C} + D) (\bar{A} + B + C + \bar{D}) (\bar{A} + B + \bar{C} + D)$$

		CD			
		00	01	11	10
A B	00		0		0
	01		0		0
	11				0
	10		0		0



		CD			
		00	01	11	10
A B	00	1	0	1	0
	01	1	0	1	0
	11	1	1	1	0
	10	1	0	1	0

Quiz 2 : Solution

Simplify the Product-Of-Sums Boolean expression below, providing a **result** in **SOP** form

$$\text{Out} = (A + B + C + \bar{D}) (A + B + \bar{C} + D) (A + \bar{B} + C + \bar{D}) (A + \bar{B} + \bar{C} + D) \\ (\bar{A} + \bar{B} + \bar{C} + D) (\bar{A} + B + C + \bar{D}) (\bar{A} + B + \bar{C} + D)$$

		CD			
		00	01	11	10
AB	00	1	0	1	0
	01	1	0	1	0
	11	1	1	1	0
	10	1	0	1	0

$$\text{Out} = \bar{C}\bar{D} + CD + ABD$$

Same Example: Both SOP and POS

$$\text{Out} = (A + B + C + \bar{D}) (A + B + \bar{C} + D) (A + \bar{B} + C + \bar{D}) (A + \bar{B} + \bar{C} + D) (\bar{A} + \bar{B} + \bar{C} + D) (\bar{A} + B + C + \bar{D}) (\bar{A} + B + \bar{C} + D)$$

A \ B	CD			
	00	01	11	10
00		0		0
01		0		0
11				0
10		0		0

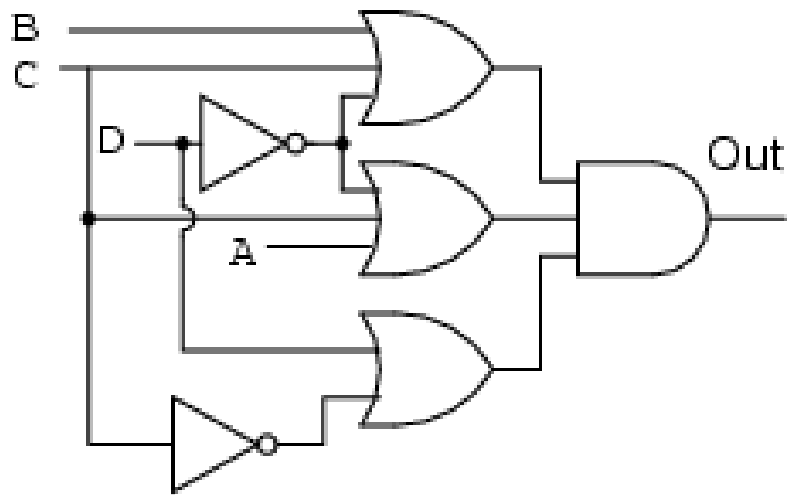
A \ B	CD			
	00	01	11	10
00	1		1	
01	1		1	
11	1	1	1	
10	1		1	

$$\text{Out} = \bar{C}\bar{D} + CD + ABD$$

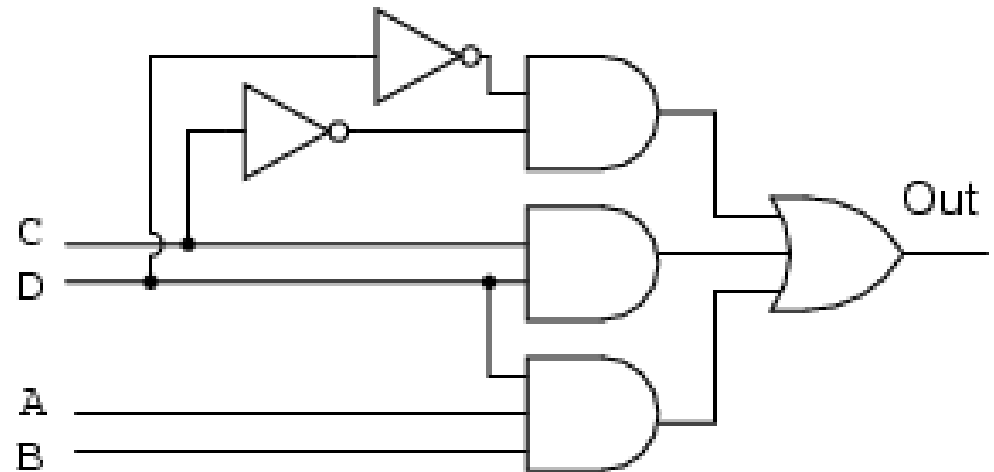
$$\text{Out} = (B + C + \bar{D}) (A + C + \bar{D}) (\bar{C} + D)$$

Implementation

$$\text{Out} = (B + C + \bar{D})(A + C + \bar{D})(\bar{C} + D)$$



$$\text{Out} = \bar{C}\bar{D} + CD + ABD$$



Don't Cares in K-map

Don't Care Conditions in K-map

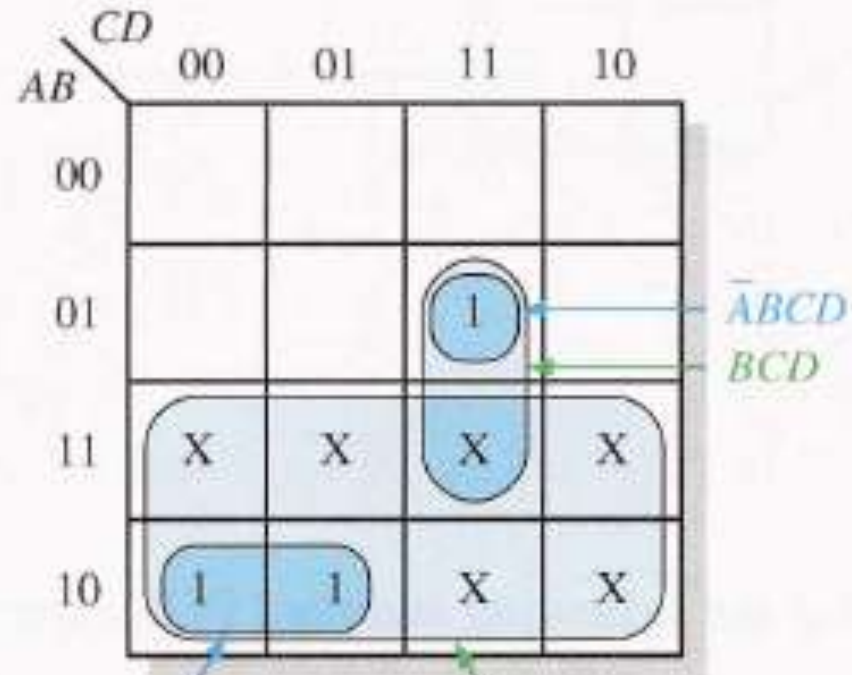
- Sometimes a situation arises in which some input variable combinations are not allowed.
- For example, in a BCD code converter, the inputs 1010, 1011, 1100, 1101, 1110 and 1111 are all **invalid**
- Since these **unallowed** states will never occur in an application involving the BCD code, they can be treated as “**don't care**” terms with respect to their effect on the output
- That is, for these “don't care” terms either a 1 or a 0 may be assigned to the output, it really does not matter since they will never occur

Example: Don't Cares in K-map

Inputs A B C D	Output Y
0 0 0 0	0
0 0 0 1	0
0 0 1 0	0
0 0 1 1	0
0 1 0 0	0
0 1 0 1	0
0 1 1 0	0
0 1 1 1	1
1 0 0 0	1
1 0 0 1	1
1 0 1 0	X
1 0 1 1	X
1 1 0 0	X
1 1 0 1	X
1 1 1 0	X
1 1 1 1	X

(a) Truth table

This circuit checks whether the input **BCD** number (**ABCD**) is more than BCD number **6 (0110)**, If yes, it **outputs (Y) a One**



Don't cares

The Y is

$$\bar{A}\bar{B}\bar{C} + \bar{A}BCD \text{ without don't cares}$$

$$A + BCD \text{ (with don't cares)}$$

Session 2.6: Summary

- Introduction to Karnaugh map
- Examples
 - SOP form
 - POS form
 - Both SOP and POS
 - Implementation
- Don't cares in K-map
 - Example